

Programmation système Linux et Optimisation système Linux

Olivia Manz, Yoann Archier

Repo github



HES-SO Master

Engineering (MSE) • Computer science

Construction de systèmes embarqués sous Linux • Jacques Supcik

Semestre printemps 2026

Contenu

1	Programmation système Linux	3
1.1	Résumé du laboratoire	3
1.2	Système de fichiers	3
1.3	Multiprocessing et Ordonnanceur	5
1.3.1	Exercice 1 - Processus, signaux et communication	5
1.3.2	Exercice 2 - CGroups	6
1.3.3	Exercice 3	7
1.4	Synthèse sur ce qui a été appris/exercé	10
1.5	Remarques et choses à retenir	10
1.6	Feedback personnel sur le laboratoire	10
2	Outils d'analyse de performance pour Linux	11
2.1	Résumé du laboratoire	11
2.2	Exercice 01	11
2.3	Exercice 02	14
2.3.1	Analyse du code source	14
2.3.2	Mesure du temps d'exécution	14
2.3.3	Optimisation	14
2.3.4	Mesures	14
2.4	Parsing de logs apache	15
2.4.1	Optimisation algorithmique	15
2.5	Mesure de la latence et de la gigue (jitter)	16
2.6	Synthèse sur ce qui a été appris/exercé	17
2.7	Remarques et choses à retenir	17
2.8	Feedback personnel sur le laboratoire	17

1 | Programmation système Linux

1.1 Résumé du laboratoire

La carte NanoPi NEO Plus2 est équipée de deux LEDs dont une verte qui donne des indications d'état (status). Le but de la première partie du laboratoire (Système de fichiers) est d'optimiser une application qui contrôle la fréquence de clignotement de cette LED à l'aide des trois boutons-poussoirs.

Le bouton K1 permet de diminuer la fréquence de clignotement, le bouton K2 permet de réinitialiser la fréquence à la valeur par défaut, et le bouton K3 permet d'augmenter la fréquence de clignotement. Une fonctionnalité optionnelle permet l'incrément/décément automatique de la fréquence de clignotement lors d'un appui prolongé sur les boutons K1 et K3.

Tous changements de fréquence doivent être indiqués dans les logs du système à l'aide de syslog.

Une contrainte supplémentaire impose l'utilisation du multiplexage des entrées et sorties pour la gestion des interactions avec les boutons et la LED. Cette exigence s'inscrit dans l'objectif d'optimisation de l'application afin d'éviter une utilisation de 100 % d'un cœur processeur, observée dans la version initiale.

La deuxième partie du laboratoire (Multiprocessing et Ordonnanceur) permet d'exercer la communication interprocessus, la gestion des signaux et le contrôle des ressources à l'aide des CGroups à l'aide de plusieurs exercices.

1.2 Système de fichiers

Nous commençons par ajouter la possibilité de modifier la fréquence de clignotement à l'aide des boutons. Afin d'identifier les GPIO associés aux boutons K1, K2 et K3 ainsi qu'à la LED verte, il est nécessaire de monter le système de fichiers debugfs afin d'accéder aux informations correspondantes, comme illustré ci-dessous.

Nous développons une application qui utilise ces GPIO pour contrôler la fréquence de clignotement de la LED en fonction des boutons pressés.

```
# mount -t debugfs none /sys/kernel/debug
cat /sys/kernel/debug/gpio
gpiochip1: GPIOs 0-223, parent: platform/1c20800.pinctrl, 1c20800.pinctrl:
gpio-0   (                | k1                ) in  lo  IRQ
gpio-2   (                | k2                ) in  lo  IRQ
gpio-3   (                | k3                ) in  lo  IRQ
gpio-10  (                | sysfs           ) out  lo
gpio-102 (                | gmac-3v3       ) out  hi
gpio-166 (                | cd             ) in  lo  IRQ ACTIVE LOW

gpiochip0: GPIOs 352-383, parent: platform/1f02c00.pinctrl, 1f02c00.pinctrl:
gpio-358 (                | vdd-cpux       ) out  hi
gpio-359 (                | reset         ) out  hi ACTIVE LOW
```

Ces informations permettent d'identifier les GPIO des boutons et de la LED. Il est ensuite possible de les contrôler en écrivant dans les fichiers correspondants dans le système de fichiers virtuel **sysfs**.

Dans un premier temps, nous prenons exemple sur le code de la version initiale qui utilise le polling. Comme on peut le voir sur l'extrait du **top** ci-dessous, cette version utilise 100% du cœur 0 de notre processeur.

```
top - 00:36:16 up 35 min,  3 users,  load average: 1.00, 0.88, 0.49
Tasks: 103 total,   2 running, 101 sleeping,   0 stopped,   0 zombie
%Cpu0  : 18.2/81.8 100[||||||||||||||||||||||||||||||||] %Cpu1  :   0.7/0.0   1[  ]
%Cpu2  :   0.0/0.0   0[  ] %Cpu3  :   0.0/0.0   0[  ]
GiB Mem : 9.1/0.5 [  ]
GiB Swap: 0.0/0.0 [  ]
PID USER  PR  NI    VIRT    RES  %CPU  %MEM     TIME+ S COMMAND
1  root   20   0    2.6m    0.3m   0.0   0.1   0:01.78 S init
...
280 root   20   0    1.7m    0.2m  99.4   0.0   0:50.28 R `-. ./build/silly_led_control
...
```

Nous ajoutons la gestion des boutons à l'aide du multiplexage des entrées et sorties avec la fonction **epoll()**. Cela nous permet de travailler de manière événementielle, en réagissant uniquement lorsque les boutons sont pressés ou lorsque le timer pour le clignotement de la LED expire, plutôt que de faire du polling constant qui consomme beaucoup de ressources CPU. Dans cette version optimisée, nous avons réussi à réduire l'utilisation du CPU à environ 1% lors de l'exécution de l'application, ce qui est une amélioration significative par rapport à la version initiale.

```
top - 01:59:55 up  1:59,  3 users,  load average: 0.09, 0.06, 0.30
Tasks: 103 total,   1 running, 102 sleeping,   0 stopped,   0 zombie
%Cpu0  :   0.0/0.0   0[  ] %Cpu1  :   0.0/0.0   0[  ]
%Cpu2  :   0.0/0.7   1[  ] %Cpu3  :   0.0/0.7   1[  ]
GiB Mem : 9.3/0.5 [  ]
GiB Swap: 0.0/0.0 [  ]
PID USER  PR  NI    VIRT    RES  %CPU  %MEM     TIME+ S COMMAND
1  root   20   0    2.6m    0.3m   0.0   0.1   0:01.78 S init
...
293 root   20   0    1.8m    0.2m   1.3   0.0   0:03.18 S `-. ./build/silly_led_control
...
```

Nous ajoutons également la fonctionnalité automatique d'incrément/décément de la fréquence de clignotement lors du maintien des boutons K1 ou K3. Nous créons, pour cela, un timer supplémentaire pour détecter le maintien d'un bouton, et nous ajustons la fréquence de clignotement en conséquence. La période de ce timer est hardcodée, la valeur est choisie de manière empirique. Cette méthode n'est pas optimale mais elle permet d'avoir une bonne réactivité sans être trop rapide.

1.3 Multiprocessing et Ordonnanceur

1.3.1 Exercice 1 - Processus, signaux et communication

L'exécution de l'application met en évidence la création de deux processus distincts. En effet, l'appel système `fork()` duplique le processus courant afin de créer un processus enfant possédant son propre contexte d'exécution. Nous observons cette duplication à l'aide de la commande `ps`, qui affiche simultanément le processus parent et le processus enfant :

```
ps -aux
...
root      307  0.0  0.0   1868   196 pts/1    S+   00:57   0:00 ./app
root      308  0.0  0.0   1868    80 pts/1    S+   00:57   0:00 ./app
```

L'application implémente un mécanisme de communication inter-processus basé sur `socketpair()`. Le processus enfant lit les données saisies au clavier puis transmet les messages textuels au processus parent, lequel les affiche sur la sortie standard.

L'arrêt normal de l'application est réalisé par l'envoi du message « `exit` » depuis l'entrée standard. Le parent détecte alors cette commande et termine proprement son exécution.

Afin de valider la gestion des signaux, le PID du processus parent est récupéré puis plusieurs signaux sont envoyés manuellement à l'aide de la commande `kill`. Les signaux `SIGHUP`, `SIGINT`, `SIGQUIT`, `SIGABRT` et `SIGTERM` sont correctement interceptés et ignorés conformément aux spécifications de l'exercice. Un simple message informatif est alors affiché à l'écran sans interruption du programme.

À l'inverse, l'envoi du signal `SIGILL(4)`, qui n'est pas explicitement géré par l'application, provoque l'arrêt immédiat du programme avec une erreur d'instruction illégale.

Sortie du programme	Commandes envoyées
<pre>hihihi [PARENT] reçu: hihihi [SIGNAL IGNORED] reçu SIGINT (2) [SIGNAL IGNORED] reçu SIGHUP (1) [SIGNAL IGNORED] reçu SIGINT (2) [SIGNAL IGNORED] reçu SIGQUIT (3) [SIGNAL IGNORED] reçu SIGABRT (6) [SIGNAL IGNORED] reçu SIGTERM (15) Illegal instruction</pre>	<pre># kill -1 307 # kill -2 307 # kill -3 307 # kill -6 307 # kill -15 307 # kill -4 307</pre>

1.3.2 Exercice 2 - CGroups

La configuration du noyau Linux est déjà adaptée à l'utilisation des cgroups. Nous créons le script **run.sh** qui monte les cgroups, il est disponible et commenté dans les sources.



1. Quel effet a la commande `echo > ...` sur les cgroups ?

La commande **echo \$\$** retourne le PID du shell courant, dans notre cas celui exécutant le script **run.sh**. Ce PID est ensuite écrit dans le fichier **tasks** du cgroup. Cela ajoute le processus shell au groupe de contrôle **mem**.

Ainsi, tous les processus enfants lancés depuis ce shell héritent de ce cgroup. Lorsque le script lance l'application **memtest**, celle-ci est donc exécutée dans le cgroup configuré et respecte la limite mémoire définie avec **memory.limit_in_bytes**.



2. Quel est le comportement du sous-système memory lorsque le quota de mémoire est épuisé ? Pourrait-on le modifier ? Si oui, comment ?

Dans notre cas, le noyau déclenche l'OOM killer du cgroup, qui termine notre programme. Nous observons alors un message **Killed** dans les logs.

La limite mémoire peut être modifiée via une écriture d'une valeur valide dans **memory.limit_in_bytes**. La valeur de limite peut être modifiée dynamiquement (dans un autre terminal), tant qu'elle est valide et supérieure à la consommation actuelle du cgroup. Si nous essayons d'écrire une valeur invalide, le message **sh: write error: Invalid argument** s'affiche. Si la nouvelle limite est inférieure à la consommation actuelle, la commande échoue avec une erreur **Device or resource busy**. Les fichiers du système cgroup ne sont pas des fichiers physiques mais des interfaces exposées par le noyau Linux ; ils ne peuvent donc pas être supprimés avec **rm**. Pour supprimer la limitation mémoire, il est possible d'utiliser la valeur **-1**.

Le comportement peut être également modifié via la configuration de l'OOM. Pour désactiver l'OOM-killer, nous pouvons écrire **1** sur le fichier **memory.oom_control**

Le comportement OOM est visible avec la commande suivante:

```
# cat /sys/fs/cgroup/memory/mem/memory.oom_control
oom_kill_disable 0
under_oom 0
oom_kill 7
```

- « **oom_kill_disable** » : indique si l'OOM killer est désactivé,
- « **under_oom** » : indique si le cgroup est en état de manque de mémoire,
- « **oom_kill** » : compteur du nombre de processus tués par OOM.

À noter que ce mécanisme appartient aux cgroups v1 et est marqué comme déprécié, mais il est encore utilisé dans le cadre de ce laboratoire.



3. Est-il possible de surveiller/vérifier l'état actuel de la mémoire ? Si oui, comment ?

Pour connaître l'utilisation de la mémoire de notre sous groupe nous utilisons la commande **# cat /sys/fs/cgroup/memory/mem/memory.usage_in_bytes** La lecture du fichier **memory.stat** fournit des informations plus détaillées sur la répartition de la mémoire (RSS, cache, pages mappées, etc.).

Source: <https://docs.kernel.org/admin-guide/cgroup-v1/memory.html>

1.3.3 Exercice 3

Nous créons un programme pour consommer du CPU et nous ajoutons des limitations concernant le CPU dans les cgroups.

```
$ mkdir /sys/fs/cgroup/cpuset
$ mount -t cgroup -o cpu,cpuset cpuset /sys/fs/cgroup/cpuset
# création de 2 groupes high et low
$ mkdir /sys/fs/cgroup/cpuset/high
$ mkdir /sys/fs/cgroup/cpuset/low
$ echo 3 > /sys/fs/cgroup/cpuset/high/cpuset.cpus
$ echo 0 > /sys/fs/cgroup/cpuset/high/cpuset.mems
$ echo 2 > /sys/fs/cgroup/cpuset/low/cpuset.cpus
$ echo 0 > /sys/fs/cgroup/cpuset/low/cpuset.mems
```



1. Les 4 dernières lignes sont obligatoires pour que les prochaines commandes fonctionnent correctement. Pouvez-vous en donner la raison ?

echo 3 > /sys/fs/cgroup/cpuset/high/cpuset.cpus et **echo 2 > /sys/fs/cgroup/cpuset/low/cpuset.cpus** Ces commandes définissent les CPU autorisés pour chaque cgroup. Le groupe high est limité au CPU 3 et le groupe low au CPU 2.

\$ echo 0 > /sys/fs/cgroup/cpuset/high/cpuset.mems Le paramètre **cpuset.mems** correspond à liste des noeuds mémoire autorisés pour un cgroup. Un noeud mémoire représente une zone physique de RAM. Certains systèmes linux peuvent avoir leur mémoire divisée en “noeuds” mais dans notre cas, nous disposons d'un seul noeud mémoire. Si nous ne renseignons pas ce paramètre, le noyau refuse d'associer un processus au cgroup, ce qui entraîne l'erreur suivante :

```
# echo $$ > /sys/fs/cgroup/cpuset/low/tasks
sh: write error: No space left on device
```




3. Sachant que l'attribut `cpu.shares` permet de répartir le temps CPU entre différents cgroups, comment devrait-on procéder pour lancer deux tâches distinctes sur le cœur 4 de notre processeur et attribuer 75% du temps CPU à la première tâche et 25% à la deuxième ?

`cpu.shares` contient un ratio. Voici les commandes que nous lançons:

```
# forcer les cgroups sur le même coeur (4)
echo 4 > /sys/fs/cgroup/cpuset/high/cpuset.cpus
echo 4 > /sys/fs/cgroup/cpuset/low/cpuset.cpus
# fixer la mémoire node (déjà fait normalement)
echo 0 > /sys/fs/cgroup/cpuset/high/cpuset.mems
echo 0 > /sys/fs/cgroup/cpuset/low/cpuset.mems
# fixer le ratio (le minimum est 2)
echo 6 > /sys/fs/cgroup/cpuset/high/cpu.shares
echo 2 > /sys/fs/cgroup/cpuset/low/cpu.shares
```

Nous avons constaté que en écrivant la valeur « 1 » dans « `cpu.shares` », celle-ci est forcée à « 2 ». Nous adaptons les valeurs en conséquence tout en conservant le même ratio. Le CPU 4 n'étant pas disponible sur notre système, nous utilisons le CPU 3.

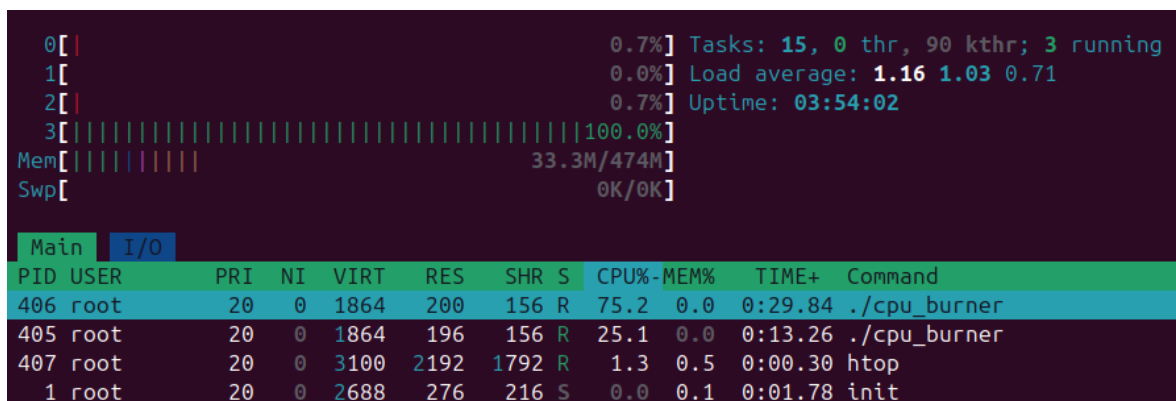


Figure 3 - Commande Htop avec répartition 75-25

1.4 Synthèse sur ce qui a été appris/exercé

Lors de la première partie de ce laboratoire, nous avons appris et exercé l'utilisation des GPIOs pour contrôler des composants matériels, en l'occurrence des boutons et une LED, ainsi que l'utilisation de `epoll()` pour faire de la programmation événementielle, ce qui est essentiel pour optimiser les applications système et éviter une utilisation excessive du CPU. Nous avons également pu ré-exercer l'utilisation de `syslog` pour enregistrer des événements dans les logs du système, ce qui est une pratique courante pour le débogage et la surveillance des applications système.

L'utilisation des timers pour gérer les fréquences de clignotement et les auto incréments/décréments a également été une partie importante de ce laboratoire, nous permettant de mieux comprendre comment gérer le temps dans les applications système.

La seconde partie nous a permis d'exercer la programmation multiprocessus avec la création de processus via `fork()`, la communication interprocessus au moyen de `socketpair()`, ainsi que la gestion et l'interception des signaux Linux. Nous étudions également la capacité, via les groupes de contrôle, à limiter l'utilisation de la mémoire et l'utilisation des CPU.

1.5 Remarques et choses à retenir

La programmation sur système Linux nécessite une bonne compréhension du système de fichiers virtuel, notamment `sysfs` et `debugfs`, pour interagir avec le matériel. L'utilisation de `epoll()` est essentielle pour créer des applications efficaces qui réagissent aux événements sans consommer inutilement des ressources CPU. Enfin, l'enregistrement d'événements dans les logs du système à l'aide de `syslog` est une pratique importante pour le débogage et la surveillance des applications système.

La gestion multiprocessus nécessite une bonne compréhension du cycle de vie des processus, de la communication interprocessus et du traitement des signaux. Les CGroups constituent un mécanisme important permettant de limiter les ressources utilisées par des processus. Ils jouent un rôle central dans la gestion des performances.

1.6 Feedback personnel sur le laboratoire

Ce laboratoire a été très instructif et didactique pour comprendre comment fonctionne la programmation système Linux, en particulier en ce qui concerne l'interaction avec le matériel à travers les GPIOs et l'optimisation des applications pour une utilisation efficace des ressources. Le fait de nous avoir donné une base de programme fonctionnelle, puis de nous demander d'ajouter des fonctionnalités tout en l'optimisant, a été une très bonne approche pour comprendre le fonctionnement global du système. Selon nous, une structure similaire pour les laboratoires précédents aurait représenté un avantage pédagogique, malgré la charge de travail supplémentaire qu'elle implique.

Concernant la seconde partie du laboratoire, les exercices proposés sont bien structurés et suffisamment courts pour permettre une bonne compréhension des différents concepts abordés. Un complément pourrait toutefois être apporté concernant les signaux Linux, notamment en établissant une correspondance entre certains événements ou actions utilisateur et les signaux générés (par exemple `Ctrl+C` pour `SIGINT` ou la fermeture d'un terminal pour `SIGHUP`).

2 | Outils d'analyse de performance pour Linux

2.1 Résumé du laboratoire

Ce laboratoire porte sur l'analyse de performances sous Linux avec **perf** : optimisation de l'utilisation du cache processeur, de la prédiction de branchement et d'un parseur de logs Apache.

2.2 Exercice 01

Nous exécutons le programme situé dans le dossier `src/06_optimization/ex01` en utilisant la commande **perf stat**.

```
# perf stat ./ex1
Performance counter stats for './ex1':
    41986.51 msec task-clock           #    1.000 CPUs utilized
         20      context-switches     #    0.476 /sec
          0      cpu-migrations        #    0.000 /sec
    48868      page-faults            #    1.164 K/sec
34260711638      cycles                #    0.816 GHz
1672170819      instructions          #    0.05  insn per cycle
269720534      branches              #    6.424 M/sec
 1011106      branch-misses          #    0.37% of all branches

42.005333145 seconds time elapsed
41.311481000 seconds user
 0.280660000 seconds sys
```

L'exécution du programme prend environ 42 secondes dont la majeure partie en espace utilisateur. Nous en déduisons qu'il travaille probablement peu avec les I/O et fait majoritairement du calcul pur et des accès mémoire. Le compteur d'instruction indique que le CPU passe le principal de son temps à attendre (0.05 insn per cycle). Le branch-misses est raisonnable (0.37%). Le programme a subi 20 changements de contexte durant son exécution, ce qui est également raisonnable.



Ce programme contient une erreur triviale qui empêche une utilisation optimale du cache. De quelle erreur s'agit-il ?

Le programme parcourt le tableau par colonnes au lieu de lignes. Il fait donc des sauts entre les adresses mémoire au lieu de modifier des emplacement mémoire contigus. En effet, le cache dépend de la localité spatiale et temporelle: il faut que 2 opérations qui utilisent une même zone mémoire soient faites proche dans le temps. Notre analyse est confirmée par la commande **perf stat -e cache-misses ./ex1**. Elle nous permet de voir que le cache misses est très élevé, le cache mémoire est donc mal utilisé.

```
# perf stat -e cache-misses ./ex1
Performance counter stats for './ex1':
 406897743      cache-misses
38.942507102 seconds time elapsed
38.265720000 seconds user
 0.276567000 seconds sys
```



Corrigez l'erreur, recompilez et mesurez à nouveau le temps d'exécution (soit avec perf stat, soit avec la commande time). Quelle amélioration constatez-vous ?

```
# perf stat ./ex1
Performance counter stats for './ex1':
    2409.50 msec task-clock           #    0.992 CPUs utilized
         17      context-switches    #    7.055 /sec
          0      cpu-migrations       #    0.000 /sec
    48867      page-faults           #   20.281 K/sec
1966007847      cycles               #    0.816 GHz
1380444909      instructions         #    0.70  insn per cycle
264953592      branches              #  109.962 M/sec
 653992      branch-misses          #    0.25% of all branches

2.429980293 seconds time elapsed
2.165846000 seconds user
0.222076000 seconds sys

# perf stat -e cache-misses ./ex1
Performance counter stats for './ex1':
 1244933      cache-misses
2.440078626 seconds time elapsed
2.123245000 seconds user
0.275689000 seconds sys
```

Compteur	Avant	Après	Amélioration
Temps d'exécution	42.0 s	2.4 s	17x plus rapide
insn per cycle	0.05	0.70	14x meilleur
cache-misses	406 897 743	1 244 933	327x moins
Instructions	1.67 Mrd	1.38 Mrd	similaire

Le nombre d'instructions est quasi identique, donc le programme fait le même travail, mais le CPU attend 14× moins. Le nombre de cache-misses est maintenant bien plus faible.



Relevez les valeurs du compteur L1-dcache-load-misses pour les deux versions de l'application. Quel facteur constatez-vous entre les deux valeurs ? **# perf stat -e L1-dcache-load-misses ./ex1**

Version initiale	Version corrigée
<pre># perf stat -e L1-dcache-load-misses ./ex1 Performance counter stats for './ex1': 406972967 L1-dcache-load-misses 40.687446478 seconds time elapsed 39.986415000 seconds user 0.288499000 seconds sys</pre>	<pre># perf stat -e L1-dcache-load-misses ./ex1 Performance counter stats for './ex1': 1297820 L1-dcache-load-misses 2.431259751 seconds time elapsed 2.186329000 seconds user 0.209635000 seconds sys</pre>

La version corrigée génère environ 300 fois moins de miss L1 que la version initiale. C'est la preuve que l'erreur était un problème de localité mémoire: le CPU devait aller chercher les données hors du cache L1 à presque chaque accès dans la version originale.



Décrivez brièvement ce que sont les événements suivants :

instructions : nombre total d'instructions exécutées par le processeur. Cet indicateur donne une idée de la quantité de travail réellement effectuée par le programme. Il est souvent comparé au nombre de cycles pour calculer l'IPC (Instructions Per Cycle).

- **cache-misses** : nombre d'accès mémoire qui n'ont pas trouvé les données dans le cache du processeur. Le CPU doit alors récupérer les données depuis un niveau de cache inférieur (L2, L3) ou directement depuis la RAM, ce qui augmente fortement le temps d'exécution.
- **branch-misses** : nombre d'erreurs de prédiction des branchements conditionnels (**if**, boucles, etc.). Les instructions dans le pipeline sont remplies en avance pour améliorer les performances. Lorsque le processeur prédit mal le chemin à suivre, il doit vider une partie du pipeline puis recommencer l'exécution, ce qui réduit les performances.
- **L1-dcache-load-misses** : nombre d'échecs lors des lectures dans le cache de données L1. Le cache L1 est le cache le plus rapide.
- **cpu-migrations** : nombre de fois où le système d'exploitation déplace un processus ou un thread d'un cœur CPU à un autre. Trop de migrations peuvent diminuer les performances car les caches doivent être reconstruits sur le nouveau cœur.
- **context-switches** : nombre de changements de contexte entre processus ou threads. Lorsqu'un changement a lieu, le système sauvegarde l'état courant puis charge celui d'une autre tâche. Un nombre élevé indique souvent beaucoup de multitâche ou d'attente sur des ressources par exemple.



Lors de la présentation de l'outil perf, on a vu que celui-ci permettait de profiler une application avec très peu d'impacts sur les performances. En utilisant la commande `time`, mesurez le temps d'exécution de notre application `ex1` avec et sans la commande `perf stat`.

```
# time perf stat ./ex1
...
real 0m 3.09s
user 0m 2.18s
sys 0m 0.28s
```

```
# time ./ex1
real 0m 2.41s
user 0m 2.17s
sys 0m 0.20s
```

On observe une augmentation du temps d'exécution réel lors de l'utilisation de **perf stat** (2.41 s contre 3.09 s). Le temps utilisateur reste pratiquement identique (2.17 s contre 2.18 s), ce qui indique que le comportement du code applicatif est peu modifié. En revanche, le temps système augmente légèrement (0.20 s contre 0.28 s), ce qui met en évidence le surcoût introduit par le profilage et la collecte des compteurs de performance.

2.3 Exercice 02

2.3.1 Analyse du code source

Le programme génère un tableau de valeurs aléatoires entre 0 et 511 puis additionne 10'000 fois toutes les valeurs supérieures ou égales à 256.

2.3.2 Mesure du temps d'exécution

Nous mesurons plusieurs fois mais les mesures sont stables à 0.01 secondes près.

```
# time ./ex2
sum=125454290000
real 0m 26.19s
user 0m 26.11s
sys 0m 0.00s
```

2.3.3 Optimisation

Après optimisation, les mesures de temps sont également stables. Nous avons observé une diminution du temps d'exécution d'environ 3 secondes.

```
# time ./ex2
sum=125454290000
real 0m 23.44s
user 0m 23.38s
sys 0m 0.00s
```

2.3.4 Mesures



À l'aide de l'outil perf et de sa sous-commande stat, en utilisant différents compteurs déterminez pourquoi le programme modifié s'exécute plus rapidement.

Version non optimisée

```
# perf stat ./ex2
sum=125454290000

Performance counter stats for './ex2':

      26381.05 msec task-clock           #    1.000 CPUs utilized
           17      context-switches     #    0.644 /sec
           0       cpu-migrations        #    0.000 /sec
          76       page-faults          #    2.881 /sec
    21526819053     cycles                #    0.816 GHz
    14768926903     instructions          #    0.69  insn per cycle
     988575602     branches              #   37.473 M/sec
    327856920      branch-misses         #   33.16% of all branches

26.392412804 seconds time elapsed

26.325403000 seconds user
  0.003968000 seconds sys
```

Version optimisée

```
# perf stat ./ex2
sum=125454290000

Performance counter stats for './ex2':

      23436.26 msec task-clock           #    0.999 CPUs utilized
           133      context-switches    #    5.675 /sec
            0      cpu-migrations        #    0.000 /sec
           107      page-faults         #    4.566 /sec
    19123431577      cycles              #    0.816 GHz
    14819330753      instructions        #    0.77  insn per cycle
    997994235       branches            #   42.583 M/sec
     873488         branch-misses       #    0.09% of all branches

23.457655803 seconds time elapsed

23.389797000 seconds user
 0.000000000 seconds sys
```

L'outil **perf** montre que le nombre de branch-misses a fortement diminué. Avant le tri, le test **if (data[i] >= 256)** est appliqué sur des valeurs aléatoires, le résultat alterne donc fréquemment entre vrai et faux. Le prédicteur de branchement du processeur ne peut alors pas anticiper correctement l'exécution et doit régulièrement vider une partie du pipeline.

Après le tri du tableau, le branchement devient plus prévisible : il reste faux pendant une longue période puis devient vrai. Le prédicteur apprend rapidement ce comportement et les erreurs deviennent rares.

Malgré le coût du `qsort()`, le tri n'est effectué qu'une seule fois alors que la boucle est répétée 10'000 fois, ce qui explique la diminution du temps d'exécution.

2.4 Parsing de logs apache



Avec les instructions précédentes, déterminez quelle fonction de notre application fait (indirectement) appel à **std::operator==<char>**.

L'option **--no-children** de la commande **perf report --no-children --demangle** masque une partie des informations hiérarchiques. En utilisant la commande **perf report -g graph --demangle**, nous pouvons développer la hiérarchie avec **+**. En remontant la callchain dans **perf report**, on observe que **std::operator==<char>** est appelée indirectement depuis **HostCounter::isNewHost()**. Cette fonction utilise **std::find()** pour vérifier si un hôte est déjà présent, ce qui entraîne de nombreuses comparaisons de chaînes.

2.4.1 Optimisation algorithmique

La solution proposée dans le codelab remplace le **std::vector** par un **std::set**. La recherche passe de $O(n)$ à $O(\log(n))$ mais l'insertion est un peu plus lente en passant de $O(1)$ à $O(\log n)$ car **set** permet une recherche dans un arbre structuré.

L'amélioration est considérable, on passe de plus de 2 minutes d'exécution à un peu plus de 2 secondes.

```
# time ./read-apache-logs access_log_NASA_Jul95_samples
Processing log file access_log_NASA_Jul95_samples
Found 14867 unique Hosts/IPs
real 2m 15.70s
user 2m 14.59s
sys 0m 0.10s
# time ./read-apache-logs-opt access_log_NASA_Jul95_samples
Processing log file access_log_NASA_Jul95_samples
Found 14867 unique Hosts/IPs
real 0m 2.27s
user 0m 1.39s
sys 0m 0.10s
```

Plutôt que d'utiliser un **set**, nous améliorons encore les performances avec un **unordered_set**: il permet des recherches et insertions en $O(1)$ moyen grâce à une table de hachage, contrairement à $O(\log n)$ pour un arbre équilibré. Puisque le gain est moins remarquable, nous utilisons un script (**tempsmoyen.sh**) pour mesurer la moyenne de temps d'exécution (hyperfine n'étant pas disponible sur notre cible). Cette dernière modification nous permet de passer de 2.273 secondes en moyenne à 2.225 secondes sur le fichier **access_log_NASA_Jul95_samples** et sur le fichier **access_log_NASA_Jul95** de 18.194 secondes à 18 secondes.

Finalement, en ajoutant encore 2 optimisations (éviter la copie de **std::string hostname** et profiter de la propriété de **insert** qui n'insère un élément que s'il n'est pas présent), nous descendons à une moyenne de 17.996 secondes pour le fichier **access_log_NASA_Jul95**.

2.5 Mesure de la latence et de la gigue (jitter)



Décrivez comment devrait-on procéder pour mesurer la latence et la gigue d'interruption, ceci aussi bien au niveau du noyau (kernel space) que de l'application (user space).

Pour mesurer la latence et la gigue d'une interruption au niveau du noyau, on peut déclencher une interruption à l'aide d'un GPIO configuré en entrée. Au début de l'ISR, activer un GPIO pour mesurer le temps entre les deux signaux grâce à un oscilloscope ou un analyseur logique. La latence correspond au temps entre le signal de requête et le signal d'exécution de l'interruption, tandis que la gigue correspond à la variation de cette latence sur plusieurs interruptions.

Pour les mesurer au niveau de l'application, nous ne pouvons pas utiliser la technique d'écrire dans **/sys/class/gpio/...** pour faire du toggling de GPIO, car cela introduit une latence supplémentaire due à l'accès au système de fichiers. En revanche il est possible d'utiliser l'appel système **mmap()** pour accéder directement aux registres du GPIO, ce qui permet de réduire la latence et d'obtenir des mesures plus précises. En utilisant **mmap()**, l'application peut directement contrôler les GPIOs sans passer par le système de fichiers, ce qui minimise les délais et permet de mesurer la latence et la gigue de manière plus précise.

2.6 Synthèse sur ce qui a été appris/exercé

Lors de ce laboratoire, nous avons utilisé différents outils d'analyse de performances sous Linux, principalement **perf**, afin de mesurer et d'identifier les causes de ralentissement dans plusieurs applications.

Dans le premier exercice, nous avons mis en évidence l'impact de la localité mémoire et de l'utilisation du cache processeur. L'analyse des compteurs de performance permet d'identifier un parcours mémoire inefficace provoquant un nombre élevé de **cache-misses**.

Le second exercice permet d'étudier l'impact de la prédiction de branchement du processeur. Nous observons qu'une simple réorganisation des données réduit fortement le nombre de **branch-misses** et améliore le temps d'exécution.

La partie consacrée au parsing des logs Apache a mis en évidence l'importance des choix algorithmiques et des structures de données. Le remplacement d'une recherche linéaire par des structures adaptées (**set**, puis **unordered_set**) permet une réduction importante du temps d'exécution.

2.7 Remarques et choses à retenir

Les compteurs matériels accessibles via **perf** permettent d'identifier plus précisément les causes d'un ralentissement, notamment les défauts de cache, les erreurs de prédiction de branchement ou les changements de contexte.

La manière de programmer, que ce soit pour le choix des structures de donnée, ou l'ordre d'accès aux données, a une énorme importance sur les performances. L'utilisation de **perf** introduit une légère surcharge d'exécution, mais peut sans autre être utilisée pour identifier les causes de lenteur dans l'exécution d'une application.

2.8 Feedback personnel sur le laboratoire

Comme précédemment, le fait d'optimiser un code existant est particulièrement intéressant et gratifiant, car cela permet de mieux visualiser l'impact que peuvent avoir des choix inadaptés de structures de données, d'algorithmes ou d'accès mémoire sur les performances globales d'une application.

Au début du laboratoire, il est indiqué qu'il est nécessaire d'ajouter plusieurs paquets dans Buildroot. Toutefois, une partie des options requises est déjà activée par défaut et la commande **perf list** ne nous a pas permis d'identifier l'absence de l'option **BR2_PACKAGE_BINUTILS_TARGET**. Ce point n'a été constaté qu'au moment de l'utilisation de **perf report**, qui nécessite la présence de l'outil **addr2line**. Afin de faciliter la procédure et d'éviter cette ambiguïté, il pourrait être intéressant d'ajouter dans le codelab une étape de vérification de la disponibilité de **addr2line** avant l'utilisation des outils **perf**.